JÖNKÖPING UNIVERSITY

JÖNKÖPING UNIVERSITY

Systemadministration – Windows

Föreläsning 6 – PowerShell

04/15/2024

Planering

Vecka	Dag	Aktivitet
13	~~Mån~~	~~**Föreläsning** – Kursintroduktion, introduktion till Windows-miljöer~~
	~~Ons~~	~~**Laboration & Redovisning laboration 1**~~
14	-	~~**Föreläsning** – Windows-klienter~~
	-	~~**Föreläsning** – Windows Server~~
	~~Fre~~	~~**Laboration & Redovisning laboration 1**~~
15	~~Mån~~	~~**Föreläsning** – Active Directory (AD)~~
	-	~~**Föreläsning** – Avancerade AD-koncept~~
	~~Fre~~	~~**Laboration & Redovisning laboration 2**~~
16	Mån	**Föreläsning** – PowerShell
	Fre	**Laboration & Redovisning laboration 3**
17	Ons	**Föreläsning** – M365, Entra ID och Azure
	Fre	**Laboration**
18	Mån	**Föreläsning** – Intune, Defender och enhetshantering
	Fre	**Laboration & Redovisning laboration 4**
19	Mån	**Föreläsning** – Patch-hantering, övervakning och omvärldsbevakning
	Ons	**Laboration & Redovisning laboration 5**
20	Mån	**Föreläsning** – Reserv
	Fre	**Laboration & Redovisning laboration 6**
21	Fre	**Pluggvecka**, extra tillfälle för **labbredovisningar**
22		**Tentamen**

Administration

- Fram till nu har vi gjort all administration via GUI
  - Server Manager
  - Active Directory Users and Computers
  - Ändring av IP-adresser
  - etc.
- Varför är administration via GUI inte alltid optimalt?

PowerShell

PowerShell (PS)

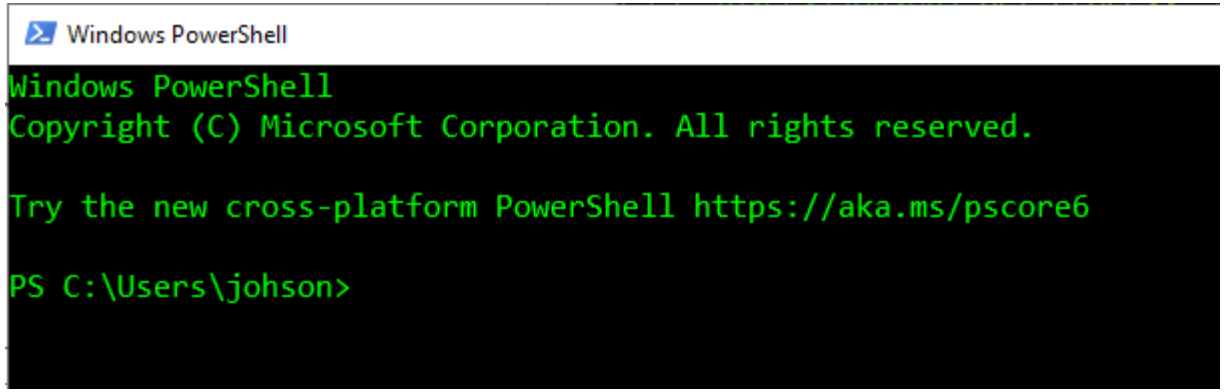
- PowerShell är en kombination av ett CLI, ett skriptingspråk och ett ramverk för automation och konfiguration
- PowerShell kan både köras lokalt och remote (**PowerShell Remoting**)
- PowerShell-filer (skript) har filändelsen .ps1
- Precis som traditionella skal (cmd, bash, etc.) har det:
  - Inbyggd hjälp (**Get-Help**)
  - Pipelines för filtrering
  - Alias för användning av välkända kommandon

Varför PowerShell?

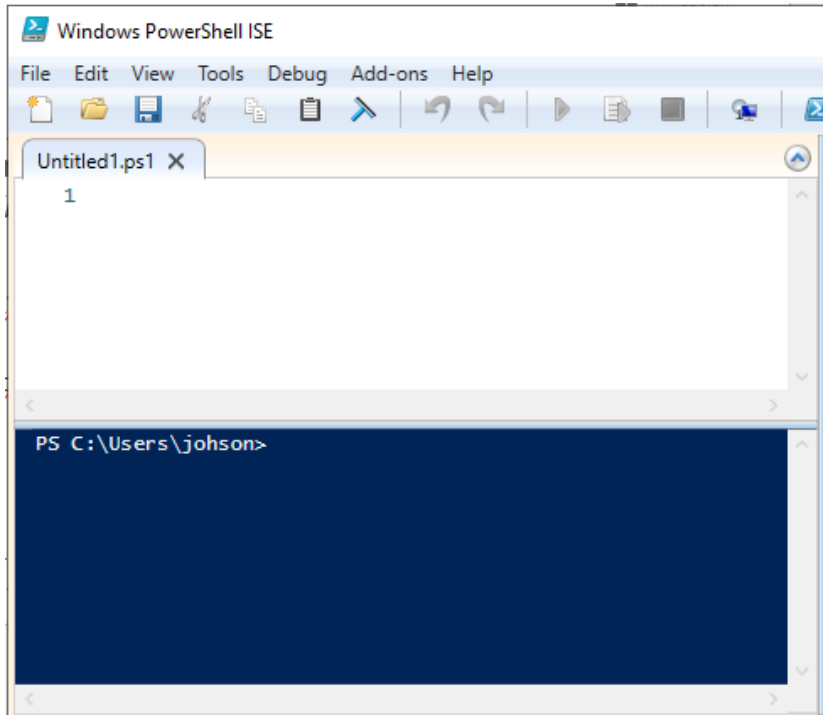
- Microsoft utgår från att den mesta administrationen av både klienter, servrar och molntjänster sker via PowerShell
- Sparar tid och förenklar arbetet, särskilt vid repetitiva uppgifter
- Hjälper till vid felsökning
- Automation av vissa uppgifter med hjälp av skript
- Minskar risken för lager 8-fel

Gränssnitt

- Interaktiva konsolen (*PowerShell console* eller *Terminal*)



- PowerShell Integrated Scripting Environment (ISE)



Cmdlets

- En viktig del är **cmdlets** (”command-lets”) som är avancerade kommandon som består av:
  - **Verb:** Get, Set, New
  - **Noun (Substantiv):** Help, Command, Service
  - **Parameter:** Inställningar för kommandot

```
PS C:\> Get-Process -Name chrome

Handles   NPM(K)  PM(K)   WS(K)   CPU(s)  Id      SI  ProcessName
-------   ------  -----   -----   ------  --      --  -----------
287       20      29804   54588   0,28    4480    1   chrome
521       26      59892   108080  13,34   7944    1   chrome
385       27      56020   112940  3,53    11868   1   chrome
```

Moduler

- PowerShell har från start ett stort antal cmdlets och andra verktyg
  - **Get-Module -ListAvailable**
- Genom att installera nya moduler (**modules**) går det att expandera funktionaliteten ytterligare
- Tänk på moduler som *optional features* som går att lägga till i PowerShell

Kommandot Get-Verb

- Beskriver alla **verb** som går att använda i cmdlets, t.ex.:
- **Get:** Hämtar ett objekt som består av ett antal värden
- **Remove:** Tar bort ett värde
- **New:** Lägger till ett nytt värde
- **Add:** Lägger till ett värde
- **Set:** Ändrar ett värde

Kommandot Get-Command

- Beskriver alla kommandon (cmdlets, funktioner och alias) som finns
- För att hitta rätt går det att söka med: **Get-Command –Noun**

```
PS C:\> Get-Command -Noun NetIPAdd*

CommandType Name                  Version   Source
----------- ----                  -------   ------
Function    Get-NetIPAddress      1.0.0.0   NetTCPIP
Function    New-NetIPAddress      1.0.0.0   NetTCPIP
Function    Remove-NetIPAddress   1.0.0.0   NetTCPIP
Function    Set-NetIPAddress      1.0.0.0   NetTCPIP
```

Alias

- Microsoft har skapat ett antal alias för ”vanliga” kommandon, t.ex. **cls**, **ls** och **cd** (stort överlapp med *cmd* samt *bash*)
- Finns alltid en motsvarande cmdlet, t.ex. är cls egentligen **Clear-Host**
- För att ta reda på ett specifikt alias skriver man **Get-Alias**

```
PS C:\> Get-Alias cd

CommandType   Name
-----------   ----
Alias         cd -> Set-Location
```

Kommandot Get-Help **&** Get-Command

- **Get-Help** är en cmdlet som är otroligt användbar för att få mer information om en cmdlet
- Med **Get-Command** kan man se alla cmdlets och andra funktioner, vill man sedan djupdyka i ett kommando skriver man:

```
PS C:\> Get-Help Get-Service
NAME
  Get-Service
SYNOPSIS
  Gets the services on a local or remote computer.
SYNTAX
  Get-Service [-ComputerName <System.String[]>] [-DependentServices] -DisplayName
<System.String[]> <...>

PS C:\> Get-Help Get-Service -Online
```

Kommandot Get-Service

→ **Get-Service** är en cmdlet för att få fram alla tjänster (**services**) som körs eller kan köras på enheten

```
PS C:\> Get-Service

Status   Name                 DisplayName
------   ----                 -----------
Stopped  AarSvc_1bc1ba        AarSvc_1bc1ba
Running  AdobeUpdateService   AdobeUpdateService
Stopped  AJRouter             AllJoyn Router Service
<…>
```

Filtrering av Get-Service

```
PS C:\> Get-Service -Name win*

Status   Name                 DisplayName
------   ----                 -----------
Running  WinDefend            Microsoft Defender Antivirus Service
Running  WinHttpAutoProx...   WinHTTP Web Proxy Auto-Discovery Se...
Running  Winmgmt              Windows Management Instrumentation
Running  WinRM                Windows Remote Management (WS-Manag...

PS C:\> Get-Service -Name win* -Exclude winrm

Status   Name                 DisplayName
------   ----                 -----------
Running  WinDefend            Microsoft Defender Antivirus Service
Running  WinHttpAutoProx...   WinHTTP Web Proxy Auto-Discovery Se...
Running  Winmgmt              Windows Management Instrumentation
```

Kommandot Get-Process

→ Är en variant på *Task Manager* fast text-baserat

```
PS C:\> Get-Process -Name chrome

Handles   NPM(K)  PM(K)   WS(K)   CPU(s)  Id      SI  ProcessName
-------   ------  -----   -----   ------  --      --  -----------
287       20      29804   54588   0,28    4480    1   chrome
521       26      59892   108080  13,34   7944    1   chrome
385       27      56020   112940  3,53    11868   1   chrome
```

→ Filtreras ofta via **pipelines**

Pipelines

- **Pipelines** är ett sätt att koppla ihop kommandon och filtrera information som bygger på användning av pipe (|)
- Vanliga kommandon att använda i en pipeline:
  - **Get-Member** För att se alla egenskaper och metoder
  - **Select-Object** För att välja ut en viss egenskap
  - **Sort-Object** För sortering av egenskaper
  - **Where-Object** För att välja ut egenskaper som har ett visst värde

Kommandot Get-Member

- Används för att få fram alla egenskaper (**properties**) och metoder (**methods**) för ett objekt (ett objekt är ett resultat av ett kommando)
- **Properties** är information om själva objektet t.ex. CPU-belastning för en viss process
  - Dessa kan exempelvis användas för att exempelvis sortera eller filtrera information
- **Methods** är en kombination instruktioner som kan genomföras på objektet
  - Dessa kan användas för att genomföra ändringar på objektet, exempelvis att avsluta en process

Kommandot Get-Member

```
PS C:\> Get-Process | Get-Member

TypeName: System.Diagnostics.Process

Name          MemberType     Definition
----          ----------     ----------
Handles       AliasProperty  Handles = Handlecount
Name          AliasProperty  Name = ProcessName
Kill          Method         void Kill()
Refresh       Method         void Refresh()
Start         Method         bool Start()
HandleCount   Property       int HandleCount {get;}
Id            Property       int Id {get;}
MachineName   Property       string MachineName {get;}
CPU           ScriptProperty System.Object CPU{get=$this.TotalProcessorTime…
<…>
```

Kommandot Select-Object (variant 1)

```
PS C:\> Get-Process | Select-Object Name, Id, CPU

Name                      Id      CPU
----                      --      ---
Adobe Crash Processor     556     60,15625
Adobe Desktop Service     22440   664,875
AdobeCollabSync           42344   10,6875
AdobeCollabSync           44192   240,046875
AdobeIPCBroker            23356   101,328125
AdobeNotificationClient   37608   0,875
AdobeUpdateService        7788
AggregatorHost            13464
ai                        8004    1,6875
ai                        9176    3,609375
ai                        29300   2,828125
amdfendrsr                4508
<…>
```

Kommandot Sort-Object

```
PS C:\> Get-Process | Select-Object Name, Id, CPU | Sort-Object -Property CPU
-Descending

Name            Id      CPU
----            --      ---
explorer        13484   8350,15625
OneDrive        13192   5929,546875
POWERPNT        28308   4492,859375
SynTPEnh        14064   2159
OUTLOOK         11684   2124,90625
ctfmon          13628   1280,640625
Dropbox         32632   1272,046875
svchost         12728   923,015625
msedgewebview2  28688   753,703125
ms-teams        23968   670,1875
<…>
```

Kommandot Select-Object (variant 2)

```
PS C:\> Get-Process | Select-Object Name, Id, CPU | Sort-Object -Property CPU -
Descending | Select-Object -First 3

Name      Id      CPU
----      --      ---
explorer  13484   8351,625
OneDrive  13192   5931,9375
POWERPNT  28308   4554,34375
```

Kommandot Where-Object

→ **Where-Object** används när man vill filtrera på ett visst värde i en **property** (egenskap)

```
PS C:\> Get-Service | Where-Object -Property Status -EQ Running

Status    Name                  DisplayName
------    ----                  -----------
Running   AdobeUpdateService    AdobeUpdateService
Running   AMD Crash Defen...    AMD Crash Defender Service
Running   AMD External Ev...    AMD External Events Utility
Running   Appinfo               Application Information
Running   AppXSvc               AppX Deployment Service (AppXSVC)
<…>
```

JÖNKÖPING UNIVERSITY

Administration i PowerShell

Hantera nätverksinställningar

- I modulen NetTCPIP finns en hel del kommandon relaterade till nätverksinställningar:
  - **Get-Command –Module NetTCPIP**
- Vanliga kommandon inkluderar:
  - **Get-NetAdapter** Hämtar information om alla nätverkskort
  - **Get-NetIPAddress** Hämtar detaljerad information om nätverkskort
  - **Remove-NetIPAddress** Tar bort adressen
  - **New-NetIPAddress** Sätter en ny IP-adress
  - **Set-NetIPAddress** Ändrar en befintlig IP-adress

Kommandot Get-NetAdapter

```
PS C:\> Get-NetAdapter

Name          InterfaceDescription   ifIndex   Status       MacAddress         LinkSpeed
----          --------------------   -------   ------       -----------        ---------
Ethernet 9    Fortinet Virtual...#2  38        Disconnected 00-09-0F-FE-XX-XX  100 Mbps
vEthernet     Hyper-V Virtual…       136       Up           00-15-5D-8B-XX-XX  10 Gbps
Ethernet 2    Realtek USB GbE…       31        Up           38-22-E2-C7-XX-XX  1 Gbps
Wi-Fi         Intel(R) Wi-Fi 6 AX200 30        Disconnected 5C-80-B6-FE-XX-XX  108 Mbps
<…>
```

Kommandot Get-NetIPConfiguration

```
PS C:\> Get-NetIPConfiguration -InterfaceIndex 31

InterfaceAlias         :   Ethernet 2
InterfaceIndex         :   31
InterfaceDescription   :   Realtek USB GbE Family Controller
NetProfile.Name        :   ad.hj.se
IPv6Address            :   2001:6b0:29:217c::249
IPv4Address            :   193.10.217.249
IPv6DefaultGateway     :   fe80::663a:eaff:feef:a082
IPv4DefaultGateway     :   193.10.217.193
DNSServer              :   2001:6b0:29:208::10
                           2001:6b0:29:208::22
                           193.10.208.10
                           193.10.208.22
```

Kommandot Get-NetAdapter **och** New-NetIPAddress

```
PS C:\> Remove-NetIPAddress -InterfaceIndex 9

Confirm Are you sure you want to perform this action? Performing operation
"Remove" on Target "NetIPAddress -IPv4Address 192.168.1.7 -InterfaceIndex 9 -
Store Active" [Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
(default is "Y"): A

PS C:\> New-NetIPAddress -InterfaceAlias Ethernet -IPAddress 192.168.1.8
-PrefixLength 24

IPAddress : 192.168.1.8
InterfaceIndex : 9
InterfaceAlias : Ethernet
AddressFamily : IPv4
Type : Unicast
PrefixLength : 24
```

Kommandot New-NetFirewallRule

- I modulen NetSecurity finns cmdlets för att hantera bland annat brandväggsregler
- En ny regel för att tillåta inkommande ping kan skapas enligt följande:

```
PS C:\> New-NetFirewallRule -DisplayName "Allow ping" -Direction Inbound
-Protocol ICMPv4 -Action Allow

Name          : {205943fc-4712-43d8-af4d-5561947679e4}
DisplayName   : Allow ping
Enabled       : True
Direction     : Inbound
Action        : Allow
<…>
```

Hantera Windows Server

- Allt som går att göra i GUI (och mer) går också att göra via PowerShell, exempelvis hantera GPO:er, hantera *Server Roles*, etc.
- Remote PowerShell (**Livelabb**)

Kommandot Get-WindowsFeature

Notera!

```
[DHCP1.ad.johson.dnlab.se]: PS C:\> Get-WindowsFeature

Display Name                                            Name                       Install State
------------                                            ----                       -------------
[ ] Active Directory Certificate Services               AD-Certificate             Available
[ ] Active Directory Domain Services                    AD-Domain-Services         Available
[ ] Active Directory Federation Services                ADFS-Federation            Available
[ ] Device Health Attestation                           DeviceHealthAttestat...    Available
[X] DHCP Server                                         DHCP                       Installed
[ ] Web Server (IIS)                                    Web-Server                 Available
<…>
```

Kommandot Install-WindowsFeature

```
[DHCP1.ad.johson.dnlab.se]: PS C:\> Install-WindowsFeature -Name Web-Server
-IncludeManagementTools

Success   Restart Needed Exit Code   Feature Result
-------   -------------- ---------   --------------
True      No             Success     {Common HTTP Features, Default Document, D...
```

```
Start Installation...
   29%
   [oooooooooooooooooooooooooooo                                                    ]
```

→ Vid lyckad installation bör nu servern kunna nås via en webbläsare

Kommandot Get-WindowsFeature **(efter inst.)**

```
[DHCP1.ad.johson.dnlab.se]: PS C:\> Get-WindowsFeature

Display Name                                            Name                       Install State
------------                                            ----                       -------------
[ ] Active Directory Certificate Services               AD-Certificate             Available
[ ] Active Directory Domain Services                    AD-Domain-Services         Available
[ ] Active Directory Federation Services                ADFS-Federation            Available
[ ] Device Health Attestation                           DeviceHealthAttestat...    Available
[X] DHCP Server                                         DHCP                       Installed
[X] Web Server (IIS)                                    Web-Server                 Installed
<…>
```

Hantera Active Directory

→ Allt som går att göra under *Active Directory Users and Computers* går också att göra via PowerShell, exempelvis:

- Skapa och ändra användarkonton
- Skapa och ändra grupper
- Skapa och ändra OU

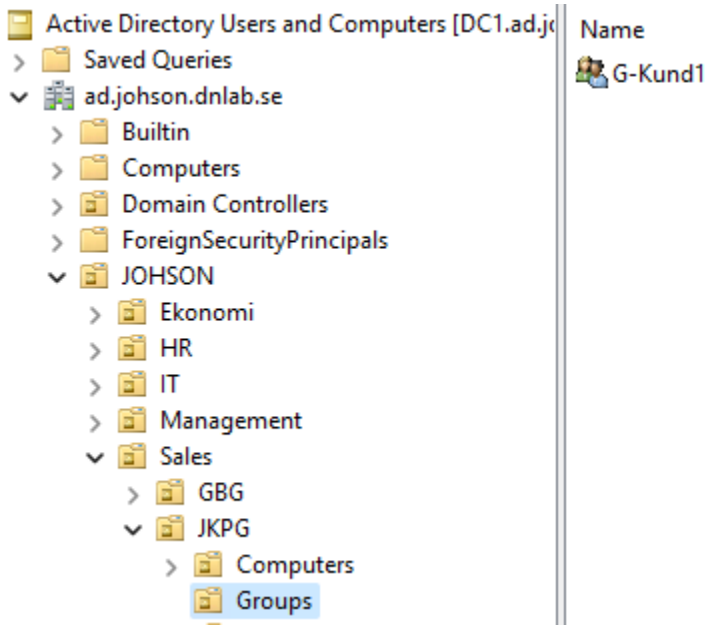
Kommandot New-ADUser **&** Add-ADGroupMember

```
[DC1.ad.johson.dnlab.se]: PS C:\> New-ADUser
-Name "Anna Johansson"
-GivenName "Anna”
-Surname "Johansson"
-SamAccountName johann
-Path "OU=Users,OU=Management,OU=JOHSON,DC=AD,DC=JOHSON,DC=DNLAB,DC=SE"
-AccountPassword (ConvertTo-SecureString "Cisco123!" -AsPlainText -Force)
-Enabled $true

[DC1.ad.johson.dnlab.se]: PS C:\> Add-ADGroupMember
-Identity "G-Management"
-Members johann
```

Kommandot New-ADGroup

```
[DC1.ad.johson.dnlab.se]: PS C:\> New-ADGroup
-Name G-Kund1
-GroupScope Global
-Path "OU=Groups,OU=JKPG,OU=Sales,OU=JOHSON,DC=AD,DC=JOHSON,DC=DNLAB,DC=SE"
```

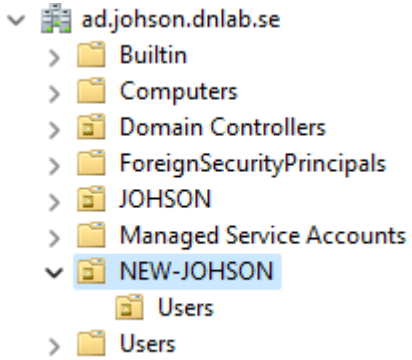


Kommandot New-ADOrganizationalUnit

```
[DC1.ad.johson.dnlab.se]: PS C:\> New-ADOrganizationalUnit -Name "NEW-JOHSON"
-Path "DC=AD,DC=JOHSON,DC=DNLAB,DC=SE”

[DC1.ad.johson.dnlab.se]: PS C:\> New-ADOrganizationalUnit -Name Users
-Path "OU=NEW-JOHSON,DC=AD,DC=JOHSON,DC=DNLAB,DC=SE"
```

→ Resultat:



→ I labbsyfte är det bra att lägga till **-ProtectedFromAccidentalDeletion $False** då rensning av felaktigt skapade OU är smidigare att ta bort

JÖNKÖPING UNIVERSITY

PowerShell-skripting

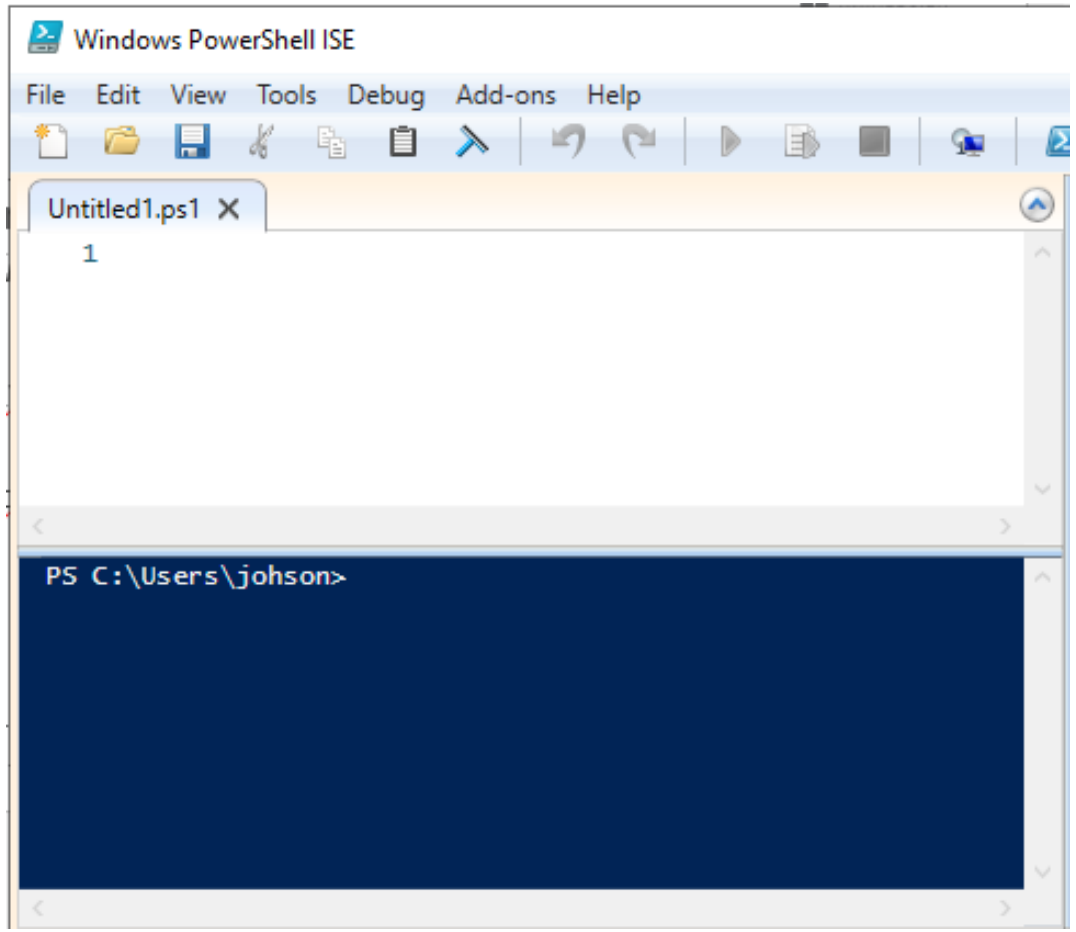
Introduktion

Skripting

- Skripting är ett antal instruktioner till en dator eller server
- Passar särskilt bra vid repetitiva uppgifter

ISE

- PowerShell Integrated Scripting Environment (ISE)
- Består av:
  - Scripting Pane
  - Command Pane
- Går att köra lokala skript på en remote enhet
  - File -> New Rmote PowerShell Tab…



PowerShell-skript

- Består av ett antal olika komponenter:
  - Variabler
  - Funktioner
  - Flow control (if/else)
  - Loopar
  - Strängar
  - Arrayer
  - Etc.

Variabler

- Är en typ av objekt där man kan lagra ett värde
- Börjar alltid med ett $
- Alla variabler måste definieras innan de används
- Finns inbyggda variabler (*Built-in variables*)
  - *$PID* eller *$Env:Computername*

```
# Skriv ett meddelande i en variabel
$meddelande = "hej världen"

# Skriv ut innehållet i variabeln via terminalen
Write-Host $meddelande
```

Variabler – User input (Read-Host)

→ Förutom att fördefiniera variabler går det få användaren av skriptet att fylla i information

```
# Skapa en variabel som tar input från användaren
$namn = Read-Host -Prompt "Skriv in ditt namn: "

# Skriv ut text där variabeln är inkluderad
Write-Output "Hej $namn! Du verkar vara duktig på att skripta!"
```

Arrays & Loops

- Listor (**arrays**) är en variant på samling (collection) där flera värden kan lagras
- Loopar (**loops**) är en samling instruktioner som ska genomföras flera gånger, men med olika ingångsvärden, t.ex. **foreach**

```
# Skapa en array med flera enheter, lägg arrayen i variabeln $servers
$servers = @(“Gitea", “PRTG")

# Skapa en loop som stänger av varje dator i arrayen och skickar ett meddelande
foreach ($server in $servers) {
    Write-Host "Turning off $server...”
    Stop-Computer -ComputerName $server -Force
}
```

Flow control / branching

- **Flow control** låter ett skript ta beslut baserat på en viss information
- Tillämpas med if/else-uttryck

```
# Definiera vilken tjänst som ska kollas
$serviceName = "WinRM"

# Kolla om tjänsten körs
$status = Get-Service -Name $serviceName | Select-Object -ExpandProperty Status

# Om tjänsten inte har status "Running" ska den startas
if ($status -ne "Running") {
    Write-Host "The $serviceName service is not running. Starting it now...”
    Start-Service -Name $serviceName
    Write-Host "The $serviceName service has been started."
} else {
    Write-Host "The $serviceName service is already running."
}
```

Notera

→ För att kunna vissa saker (t.ex. ändra inställningar, stänga av enheter, med mera) remote via PowerShell måste datorn (t.ex. PrivClient) ändra *ExecutionPolicy*

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Exempel-skript (Skapa upp en OU-struktur)

```
# Definiera namn på grund-OU
$rootOUName = "JOHSON-TEST"

# Definiera namn på child-OU
$childOUs = @("Users", "Groups", "Computers", "Printers", "Servers")

# Definiera distinguished name för domänen
$domainDN = "DC=ad,DC=johson,DC=dnlab,DC=se"

# Skapa grund-OU
New-ADOrganizationalUnit -Name $rootOUName -Path $domainDN

# Skapa child-OUs under grund-OUs
foreach ($ou in $childOUs) {
    $ouPath = "OU=$rootOUName,$domainDN"
    New-ADOrganizationalUnit -Name $ou -Path $ouPath
}

Write-Host "OU structure created successfully."
```

Livelabbar

- **Livelabb:** Skapa upp en OU-struktur (exempel-skript på föregående sida) från PrivClient (med PowerShell remoting)
- **Livelabb:** PowerShell & ChatGPT, https://chat.openai.com/
- **Livelabb:** Shutdown-skript från PrivClient

Inför kommande laborationer

- Gruppera er i en grupp om två studenter (i undantagsfall kan tre fungera)
- Det finns 33 grupper med namnen:
  - M365-tenant 1
  - M365-tentant 2
  - M365-tentant X
- Ska vara klart imorgon tisdag kl. 13.00 (annars blir ni automatiskt placerade)
- Första studenten som går med i en grupp blir gruppledare (den vars domän kommer bli en M365-tenant)

kahoot.it

JÖNKÖPING UNIVERSITY